

Chapter 4: Structures and Union

अध्याय 4: स्ट्रक्चर एवं यूनियन

UP Board Class 10 – Computer (836) | Papers: 2023 (05000/226), 2024 (40000/4031), 2025 (L/1076), 2026 (L/S-1027, 836 UA)

2022 paper not uploaded, so not included / 2022 का पेपर अपलोड नहीं था

Q1 | 2026 | Set: L/S-1027 (836 UA) | Part-A MCQ 11 | 1 Mark

हि : यदि आपको 50 कर्मचारियों का नाम और वेतन स्टोर करना हो, तो C की कौन-सी सुविधा सबसे उपयुक्त होगी ? (A) केवल पूर्णांकों की एक ऐरे (B) स्ट्रक्चर (C) प्वाइंटर (D) केवल कैरेक्टर ऐरे

En: To store the names and salaries of 50 employees, which C feature is most appropriate? (A) Single array of integers (B) Structure (C) Pointer (D) Character array only

Answer: (B) Structure / स्ट्रक्चर

हि : नाम (char) और वेतन (float) अलग-अलग डेटा टाइप हैं। ऐरे में केवल एक ही टाइप के तत्व रह सकते हैं, इसलिए दोनों को एक रिकॉर्ड में रखने के लिए स्ट्रक्चर चाहिए, और 50 कर्मचारियों के लिए स्ट्रक्चर की ऐरे:

En: Name (char) and salary (float) are different data types. An array holds only one type, so a structure is needed to hold both in one record, and an array of structures for 50 employees:

```
struct employee {
    char name[30];
    float salary;
};
struct employee emp[50];
```

Q2 | 2024 | Set: 836 (40000/4031) | Part-A MCQ 10 | 1 Mark

हि : C स्ट्रक्चर के बारे में सही कथन चुनें: (A) यह रैन्डम फ्री मेमोरी लोकेशन पर संग्रहित होते हैं (B) यह रजिस्टर मेमोरी लोकेशन में संग्रहित होते हैं (C) यह कन्टीग्यूस मेमोरी लोकेशन में संग्रहित होते हैं (D) इनमें से कोई नहीं

En: Choose a correct statement about C structure: (A) stored on random free memory location (B) stored in register memory location (C) stored in contiguous memory location (D) None of these

Answer: (C) It is stored in contiguous memory location / कन्टीग्यूस (लगातार) मेमोरी में

हि : स्ट्रक्चर के सभी सदस्य मेमोरी में एक के बाद एक लगातार स्थानों पर रखे जाते हैं (कंपाइलर आवश्यकता अनुसार थोड़ा padding जोड़ सकता है)। स्ट्रक्चर का कुल आकार सभी सदस्यों के आकार के योग के बराबर या उससे थोड़ा अधिक होता है।

En: All members of a structure are placed one after another in contiguous memory (the compiler may add some padding). The total size is the sum of its members' sizes, or slightly more.

Q3 | 2024 | Set: 836 (40000/4031) | Part-B (VSA) Q7 | 1 Mark

हि : स्ट्रक्चर क्या होते हैं ?

En: What is structure?

Answer / उत्तर

हि : स्ट्रक्चर एक यूजर डिफाईंड डेटा टाइप है जो अलग-अलग डेटा टाइप के वेरिएबल्स को एक नाम के अंतर्गत समूहित करता है। इसे struct कीवर्ड से बनाते हैं।

En: A structure is a user-defined data type that groups variables of different data types under one name. It is created using the struct keyword.

```
struct student {
    int roll;
    char name[20];
    float marks;
};
```

Q4 | 2025 | Set: L/1076 | Part-B (VSA) Q7 | 1 Mark

हि : 'C' में स्ट्रक्चर और यूनियन के बीच क्या अंतर है ?

En: What is the difference between structure & union in 'C'?

Answer / उत्तर

Structure / स्ट्रक्चर	Union / यूनियन
Declared with struct / struct कीवर्ड से	Declared with union / union कीवर्ड से
Each member has separate memory / हर सदस्य की अलग मेमोरी	All members share the same memory / सभी सदस्य एक ही मेमोरी साझा करते हैं
Size = sum of all members / आकार = सभी सदस्यों का योग	Size = largest member / आकार = सबसे बड़े सदस्य के बराबर
All members can hold values at once / सभी सदस्यों में एक साथ मान	Only one member holds a valid value at a time / एक समय में केवल एक सदस्य का मान मान्य

Extra Revision Notes / अतिरिक्त पुनरावृत्ति नोट्स (not from papers / पेपर से नहीं)

Structure: syntax, access and example / स्ट्रक्चर : सिंटैक्स, एक्सेस, उदाहरण

हि : स्ट्रक्चर के सदस्य को डॉट (.) ऑपरेटर से एक्सेस करते हैं (पॉइंटर से हो तो ->) ।

En: Members are accessed with the dot (.) operator (-> when using a pointer).

```
#include <stdio.h>
struct student { int roll; char name[20]; float marks; };
int main()
{
    struct student s1 = {1, "Ravi", 85.5};
    printf("%d %s %.1f", s1.roll, s1.name, s1.marks);
    return 0;
}
// Output: 1 Ravi 85.5
```

Union: syntax and example / यूनियन : सिंटैक्स और उदाहरण

```
#include <stdio.h>
union data { int i; float f; char c; };
int main()
{
    union data d;
    d.i = 10;
    printf("%d\n", d.i);           // 10
    printf("%zu", sizeof(union data)); // 4 (largest member)
    return 0;
}
```

हि : d.f में मान डालने पर d.i का पुराना मान मिट जाता है क्योंकि दोनों एक ही मेमोरी में हैं ।

En: Storing a value in d.f overwrites d.i because both share the same memory.